

Formalización de la confluencia $\beta\eta$ en Lean

Maximiliano Onofre Martínez

Tabla de contenidos

Introducción	1
1 Relaciones	2
1.1. Definiciones básicas	2
2 Cálculo λ	6
2.1. La representación <i>locally nameless</i>	6
3 Confluencia $\beta\eta$	10
3.1. Confluencia fuerte de η	10

Introducción

1 Relaciones

1.1. Definiciones básicas

Definición 1.1.1. Una relación binaria en un tipo X es una función $X \rightarrow X \rightarrow \text{Prop}$, es decir, una proposición sobre pares de elementos de X .

```
def relation (X : Type) := X → X → Prop
```

Definición 1.1.2. Sea r una relación en A . Entonces r induce las siguientes relaciones binarias.

I. $\text{ReflGen } r$ es la cerradura reflexiva de r , definida inductivamente de la siguiente manera:

```
inductive ReflGen {A : Type} (r : relation A) : relation A
| refl {a : A} : ReflGen r a a
| single {a b : A} : r a b → ReflGen r a b
```

II. $\text{ReflTransGen } r$ es la cerradura reflexiva y transitiva de r , definida inductivamente de la siguiente manera:

```
inductive ReflTransGen {A : Type} (r : relation A) : relation A
| refl {a : A} : ReflTransGen r a a
| tail {a b c : A} : ReflTransGen r a b → r b c → ReflTransGen r a c
```

De ahora en adelante escribimos *variable* $\{A : \text{Type}\}$, para evitar la repetición de $\{A : \text{Type}\}$. Mientras que en los diagramas escribimos $\rightarrow$ para r a b , $\rightarrow^=$ para $\text{ReflGen } r$ a b y $\twoheadrightarrow$ para $\text{ReflTransGen } r$ a b .

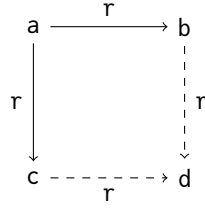
Definición 1.1.3. Sea r una relación en A .

I. $\text{Join } r$ es una nueva relación en la que dos términos están relacionados si existe un tercer término con el que ambos se relacionen.

```
def Join (r : relation A) : relation A := fun x y ↦ ∃ z, r x z ∧ r y z
```

II. Una relación tiene la *propiedad del diamante* cuando todas las reducciones con un origen común se pueden unir.

1.1. Definiciones básicas

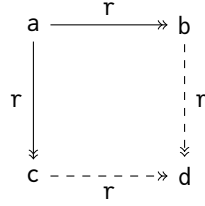


```
def Diamond (r : relation A) := ∀ {a b c : A}, r a b → r a c → Join r b c
```

Confluencia y Conmutación

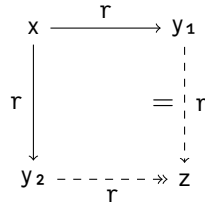
Definición 1.1.4. Sea r una relación en A .

I. r es **confluente** si su cerradura reflexiva y transitiva tiene la propiedad del diamante.



```
def Confluent (r : relation A) := Diamond (ReflTransGen r)
```

II. r es **fuertemente confluente** si:



```
def StronglyConfluent (r : relation A) :=
  ∀ {x y1 y2}, r x y1 → r x y2 → ∃ z, ReflGen r y1 z ∧ ReflTransGen r y2 z
```

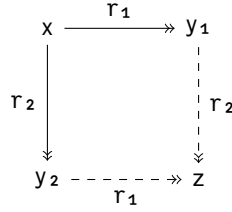
Teorema 1.1.1. Si una relación es fuertemente confluente, entonces también es confluente.

1.1. Definiciones básicas

theorem *StronglyConfluent.toConfluent* ($h : \text{StronglyConfluent } r$) : *Confluent* r

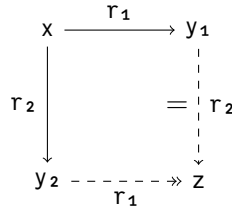
Definición 1.1.5. Sean r_1 y r_2 dos relaciones en A .

I. r_1 y r_2 **conmutan** si:



```
def Commute (r1 r2 : relation A) :=
  ∀ {x y1 y2}, ReflTransGen r1 x y1 → ReflTransGen r2 x y2 →
    ∃ z, ReflTransGen r2 y1 z ∧ ReflTransGen r1 y2 z
```

II. r_1 y r_2 **conmutan fuertemente** si:



```
def StronglyCommute (r1 r2 : relation A) :=
  ∀ {x y1 y2}, r1 x y1 → r2 x y2 → ∃ z, ReflGen r2 y1 z ∧ ReflTransGen r1 y2 z
```

Teorema 1.1.2. Si dos relaciones conmutan fuertemente, entonces también conmutan.

theorem *StronglyCommute.toCommute* { $r_1 r_2 : \text{relation } A$ } ($h : \text{StronglyCommute } r_1 r_2$) :
Commute $r_1 r_2$

Teorema 1.1.3. La conmutación entre relaciones es simétrica, es decir, si r_1 y r_2 conmutan, entonces r_2 y r_1 también.

theorem *Commute.symmetric* : *Symmetric* (@*Commute* A)

1.1. Definiciones básicas

Teorema 1.1.4 (Hindley-Rosen). *Sean r_1 y r_2 dos relaciones sobre un mismo tipo. Si ambas son confluente y conmutan entre sí, entonces su unión $r_1 \sqcup r_2$ es confluente.*

```
theorem Commute.join_confluent { $r_1$   $r_2$  : relation A}  
  ( $c_1$  : Confluent  $r_1$ ) ( $c_2$  : Confluent  $r_2$ ) ( $comm$  : Commute  $r_1$   $r_2$ ) :  
    Confluent ( $r_1 \sqcup r_2$ )
```

2 Cálculo λ

2.1. La representación *locally nameless*

Definición 2.1.1. Utilizamos una representación *locally nameless* para el cálculo λ , donde las variables ligadas se representan como números naturales (índices de De Bruijn) y las variables libres como términos del tipo `Var`. El tipo `Var` representa nombres: la igualdad es decidible en ellos (`DecidableEq`) y es posible generar uno nuevo para cualquier conjunto finito dado de nombres (`HasFresh`).

```
universe u
variable {Var : Type u} [HasFresh Var] [DecidableEq Var]

inductive Term (Var : Type u)
| bvar : ℕ → Term Var
| fvar : Var → Term Var
| abs  : Term Var → Term Var
| app  : Term Var → Term Var → Term Var
```

Por ejemplo, podemos representar la expresión “ $\lambda x. x y$ ” de la siguiente manera.

```
variable {y : Var}
def ejemplo_rep := abs (app (bvar 0) (fvar y))
```

Lo cual también se puede escribir como “ $\lambda. 0 y$ ”.

Definición 2.1.2. La sustitución consiste en reemplazar una variable libre por un término.

```
def subst (m : Term Var) (x : Var) (sub : Term Var) : Term Var :=
  match m with
  | bvar i   => bvar i
  | fvar x'  => if x = x' then sub else fvar x'
  | app l r  => app (l.subst x sub) (r.subst x sub)
  | abs M    => abs <| M.subst x sub
```

2.1. La representación *locally nameless*

Definimos una notación para la sustitución de variables libres que imita la notación matemática estándar.

```
scoped notation t:max "[" x " := " t' "]" => subst t x t'
```

Definición 2.1.3. La función `fv`, definida a continuación, calcula el conjunto de variables libres de una expresión. Dado que utilizamos una representación *locally nameless*, donde las variables ligadas se representan como índices, cualquier nombre que veamos es una variable libre de un término.

```
def fv : Term Var → Finset Var
| bvar _ => {}
| fvar x => {x}
| abs e1 => e1.fv
| app l r => l.fv ∪ r.fv
```

El tipo `Finset Var` representa un conjunto finito de elementos del tipo `Var`.

Definición 2.1.4. La operación de apertura sustituye un índice por un término.

```
def openRec (i : ℕ) (sub : Term Var) : Term Var → Term Var
| bvar i' => if i = i' then sub else bvar i'
| fvar x  => fvar x
| app l r => app (openRec i sub l) (openRec i sub r)
| abs M   => abs <| openRec (i+1) sub M
```

También definimos una notación para `openRec`.

```
scoped notation:68 e "[" i " ~> " sub "]" => openRec i sub e
```

Las aplicaciones habituales de esta función sustituyen el índice cero por una variable o un término λ . La siguiente definición es una forma práctica de indicar estos usos.

```
def open' {X} (e u) := @openRec X 0 u e

scoped infixr:80 " ^ " => open'
```

Así, $(e \wedge x)$ puede leerse como “reemplaza el índice 0 por la variable x en e ” o “abre e con la variable x ”.

2.1. La representación *locally nameless*

Definición 2.1.5 (Local closure). Un término se denomina “localmente cerrado” cuando no contiene índices no ligados. La proposición LC se cumple cuando una expresión e es localmente cerrada.

```
inductive LC : Term Var → Prop
| fvar (x : Var) : LC (fvar x)
| abs (L : Finset Var) (e : Term Var) : (∀ x ∉ L, LC (e ^ fvar x)) → LC (abs e)
| app {l r} : l.LC → r.LC → LC (app l r)
```

Definición 2.1.6. Dada una relación r en el tipo $Term Var$, se llama cerradura compatible de r a la relación $CompatGen$ r , definida de la siguiente manera.

```
inductive CompatGen (r : Term Var → Term Var → Prop) : Term Var → Term Var → Prop
| base : r M N → CompatGen r M N
| appL : LC Z → CompatGen r M N → CompatGen r (app M Z) (app N Z)
| appR : LC Z → CompatGen r M N → CompatGen r (app Z M) (app Z N)
| xi (L : Finset Var) : (∀ x ∉ L, CompatGen r (M ^ fvar x) (N ^ fvar x)) →
  CompatGen r (abs M) (abs N)
```

Definición 2.1.7. Sea $Beta$ una relación en $Term Var$, definida de la siguiente manera.

```
inductive Beta : Term Var → Term Var → Prop
| beta : LC (abs A) → LC B → Beta (app (abs A) B) (A ^ B)
```

I. La reducción β de un solo paso es la cerradura compatible de $Beta$.

```
@[reduction_sys "β"]
abbrev FullBeta : Term Var → Term Var → Prop := CompatGen Beta
```

II. La reducción β es la cerradura reflexiva y transitiva de $CompatGen$ $Beta$, es decir, $ReflTransGen$ $FullBeta$.

Definición 2.1.8. Sea Eta una relación en $Term Var$, definida de la siguiente manera.

```
inductive Eta : Term Var → Term Var → Prop
| eta : LC M → Eta (abs (app M (bvar 0))) M
```

I. La reducción η de un solo paso es la cerradura compatible de Eta .

2.1. La representación *locally nameless*

```
@[reduction_sys "ηf"]  
abbrev FullEta : Term Var → Term Var → Prop := CompatGen Eta
```

II. La reducción η es la cerradura reflexiva y transitiva de `CompatGen Eta`, esto es `ReflTransGen FullEta`.

Teorema 2.1.1. *El lado derecho de una reducción η es localmente cerrado.*

```
theorem step_lc_r (step : M →ηf M') : LC M'
```

3 Confluencia $\beta\eta$

3.1. Confluencia fuerte de η

Teorema 3.1.1. *La reducción η de un solo paso es fuertemente confluente.*

Demostración. Supongamos que M , M_1 y M_2 son términos lambda tal que $M \rightarrow_{\eta^f} M_1$ y $M \rightarrow_{\eta^f} M_2$. Entonces existe un término lambda z tal que $\text{ReflGen FullEta } M_1 \ z$ y $M_2 \rightarrow_{\eta^f} z$.

Código Lean

```
theorem stronglyConfluent_eta :
  StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
```

Estado de la prueba

```
M M1 M2 : Term Var
h1 : M →ηf M1
h2 : M →ηf M2
⊢ ∃ z, ReflGen FullEta M1 z
  ∧ M2 →ηf z
```

Como la cerradura reflexiva está contenida en la reflexiva y transitiva, basta con demostrar que $\exists z, \text{ReflGen FullEta } M_1 \ z \wedge \text{ReflGen FullEta } M_2 \ z$.

Por inducción sobre la definición de $M \rightarrow_{\eta^f} M_1$, demostraremos que, para toda M_2 , si $M \rightarrow_{\eta^f} M_2$, entonces existe una z tal que $\text{ReflGen FullEta } M_1 \ z$ y $\text{ReflGen FullEta } M_2 \ z$. Procedemos por casos según la última regla utilizada en la derivación de $M \rightarrow_{\eta^f} M_1$.

II. Si la última regla fue appL , entonces $M = \text{app } P \ N$ y $M_1 = \text{app } P_1 \ N$, para alguna P , P_1 y N tales que $LC \ N$ y $P \rightarrow_{\eta^f} P_1$.

```
theorem stronglyConfluent_eta : StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  :
  case appL N P P1 lc_N r_P_P1 ih =>
```

```
M M1 M2 N P P1 : Term Var
lc_N : LC N
r_P_P1 : P →ηf P1
ih : ∀ {M2 : Term Var}, P →ηf M2 → ∃ z,
  ReflGen FullEta P1 z ∧ ReflGen FullEta M2 z
h2 : app P N →ηf M2
⊢ ∃ z, ReflGen FullEta (app P1 N) z ∧ ReflGen FullEta M2 z
```

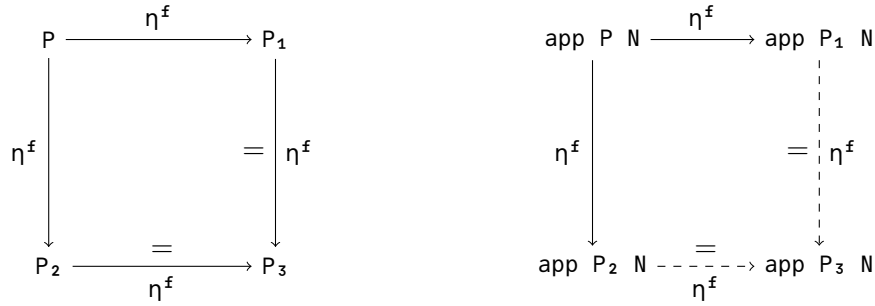
3.1. Confluencia fuerte de η

A continuación, procedemos por casos sobre la derivación de $\text{app } P \ N \rightarrow_{\eta^f} M_2$:

(a) Si la última regla fue appL , entonces $M_2 = \text{app } P_2 \ N$ tal que $P \rightarrow_{\eta^f} P_2$ y $\text{LC } N$. Por la hipótesis de inducción aplicada sobre $P \rightarrow_{\eta^f} P_2$, sabemos que existe un término P_3 tal que $\text{ReflGen FullEta } P_1 \ P_3$ y $\text{ReflGen FullEta } P_2 \ P_3$. Por lo tanto, tomamos $z = \text{app } P_3 \ N$.

```
case appL N P P1 lc_N r_P_P1 ih =>
  case appL P2 r_P_P2 _ =>
    obtain ⟨P3, refl_P1_P3, refl_P2_P3⟩ := ih r_P_P2
    use app P3 N
```

```
M M1 N P P1 P2 P3 : Term Var
lc_N : LC N
r_P_P1 : P →ηf P1
ih : ∀ {M2 : Term Var}, P →ηf M2 → ∃ z,
  ReflGen FullEta P1 z ∧ ReflGen FullEta M2 z
r_P_P2 : P →ηf P2
refl_P1_P3 : ReflGen FullEta P1 P3
refl_P2_P3 : ReflGen FullEta P2 P3
⊢ ReflGen FullEta (app P1 N) (app P3 N) ∧ ReflGen FullEta (app P2 N) (app P3 N)
```



Para cerrar cada componente de la conjunción procedemos por casos sobre $\text{ReflGen FullEta } P_1 \ P_3$ y $\text{ReflGen FullEta } P_2 \ P_3$, respectivamente: si son reflexivas, ambos lados ya coinciden; si constan de un paso, reducimos P_1 a P_3 en la primera y P_2 a P_3 en la segunda, dejando fija N , mediante la regla appL .

```
case appL N P P1 lc_N r_P_P1 ih =>
  case appL P2 r_P_P2 _ =>
    obtain ⟨P3, refl_P1_P3, refl_P2_P3⟩ := ih r_P_P2
    use app P3 N
    constructor
    · cases refl_P1_P3
      case refl => rfl
```

3.1. Confluencia fuerte de η

```

case single r_P1_P3 => exact .single (.appl lc_N r_P1_P3)
• cases refl_P2_P3
case refl => rfl
case single r_P2_P3 => exact .single (.appl lc_N r_P2_P3)

```

(b) Si la última regla fue appR, entonces $M_2 = \text{app } P \ N_1$, para alguna N_1 donde $LC \ P$ y $N \rightarrow_{\eta^f} N_1$. Así, tomamos $z = \text{app } P_1 \ N_1$.

```

case appl N P P1 lc_N r_P_P1 ih =>
  case appl P2 r_P_P2 _ =>
  :
  case appR N1 _ r_N_N1 =>

```

```

M M1 N N1 P P1 : Term Var
lc_N : LC N
r_P_P1 : P →ηf P1
ih : ∀ {M2 : Term Var}, P →ηf M2 → ∃ z,
    ReflGen FullEta P1 z ∧ ReflGen FullEta M2 z
lc_P : LC P
r_N_N1 : N →ηf N1
⊢ ReflGen FullEta (app P1 N) (app P1 N1) ∧ ReflGen FullEta (app P N1) (app P1 N1)

```

$$\begin{array}{ccc}
 \text{app } P \ N & \xrightarrow{\eta^f} & \text{app } P_1 \ N \\
 \downarrow \eta^f & & \downarrow \eta^f \\
 \text{app } P \ N_1 & \xrightarrow[\eta^f]{=} & \text{app } P_1 \ N_1
 \end{array}$$

Cada componente de la conjunción se obtiene con una sola reducción: en la primera reducimos N a N_1 dejando fija P_1 , y en la segunda reducimos P a P_1 dejando fija N_1 .

Las premisas de cerradura local que piden las reglas appR y appl se obtienen del teorema 2.1.1 (step_lc_r): de $r_P_P_1$ obtenemos $LC \ P_1$ y de $r_N_N_1$ obtenemos $LC \ N_1$.

```

case appl N P P1 lc_N r_P_P1 ih =>
  case appl P2 r_P_P2 _ =>
  :
  case appR N1 _ r_N_N1 =>

```

```

use app P1 N1
constructor
• exact .single (.appR (step_lc_r r_P_P1) r_N_N1)
• exact .single (.appL (step_lc_r r_N_N1) r_P_P1)

```

III. Si la última regla fue appR, entonces $M = \text{app } N \ P$ y $M_1 = \text{app } N \ P_1$, para alguna N , P y P_1 tales que $LC \ N$ y $P \rightarrow_{\eta^f} P_1$.

```

theorem stronglyConfluent_eta : StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  :
  case appR N P P1 lc_N r_P_P1 ih =>

```

```

M M1 M2 N P P1 : Term Var
lc_N : LC N
r_P_P1 : P →ηf P1
ih : ∀ {M2 : Term Var}, P →ηf M2 → ∃ z,
    ReflGen FullEta P1 z ∧ ReflGen FullEta M2 z
h2 : app N P →ηf M2
⊢ ∃ z, ReflGen FullEta (app N P1) z ∧ ReflGen FullEta M2 z

```

Este caso es análogo al anterior: basta intercambiar el argumento que se reduce con el que queda fijo y, con ellos, las reglas appL y appR. La formalización es la siguiente.

```

theorem stronglyConfluent_eta : StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  :
  case appR N P P1 lc_N r_P_P1 ih =>
    cases h2
    case appL N1 r_N_N1 _ =>
      use app N1 P1
      constructor
      • exact .single (.appL (step_lc_r r_P_P1) r_N_N1)
      • exact .single (.appR (step_lc_r r_N_N1) r_P_P1)
    case appR P2 r_P_P2 =>
      obtain ⟨P3, refl_P1_P3, refl_P2_P3⟩ := ih r_P_P2
      use app N P3
      constructor
      • cases refl_P1_P3
        case refl => rfl
        case single r_P1_P3 => exact .single (.appR lc_N r_P1_P3)
      • cases refl_P2_P3

```

3.1. Confluencia fuerte de η

```
case refl => rfl
case single r_P2_P3 => exact .single (.appR lc_N r_P2_P3)
```

□